Simulation of Robotic Systems Lab 03

Lätitzia Brandt

May 27, 2025

Task 1

Build a 2-DOF robot consisting of 2 rotational joints, rotating around the z-axis of the world coordinate system (WCS). You can use the snake robot and downgrade it to 2 dofs or you build from scratch. Figure 1 shows the 2-DOF robot in its zero position (all joints are zero).

You can use one of your python scripts from exercise 2 as a starting point and calculate the inverse kinematics as a separate function.

This function should provide an analytical solution (basic trigonometry). Your implementation should calculate two solutions (elbow up and down).
At the start set all joints to 20 degrees to avoid singularities.
Control your robot's TCP in Cartesian space with your keyboard (see also code in P3_Task1.py). The key 'e' should toggle elbow up and down.

Alternative (voluntary): Evaluate your result with a small GUI (adopted from exercise 1) to control the TCP in Cartesian space.

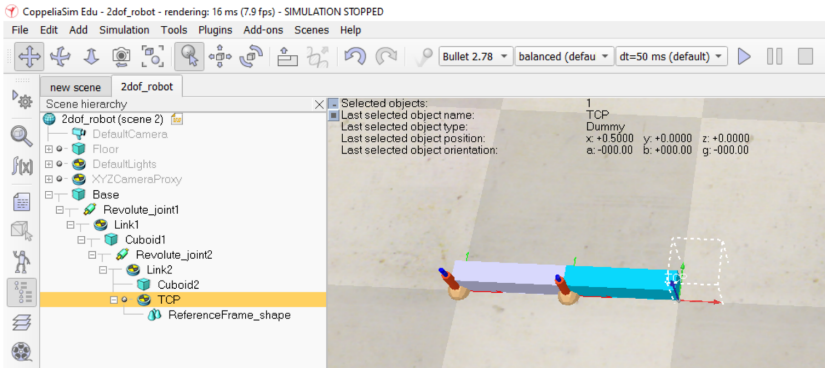
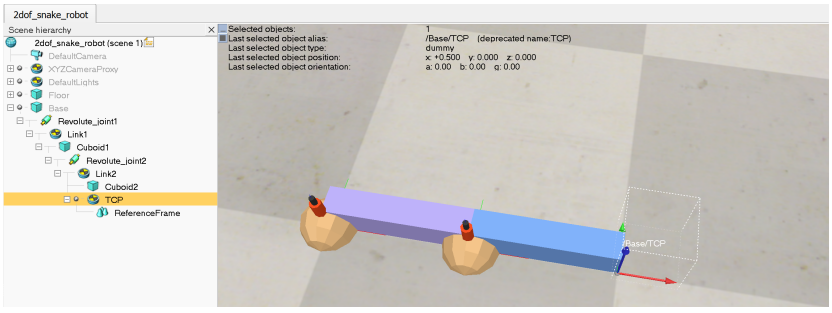


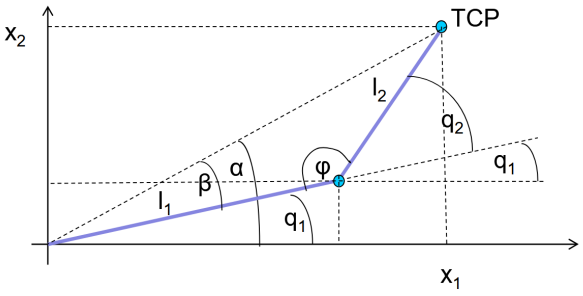
Figure 1: 2-DOF snake robot

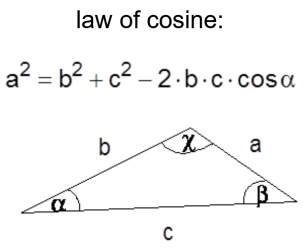
2-DOF snake robot model



Preparation

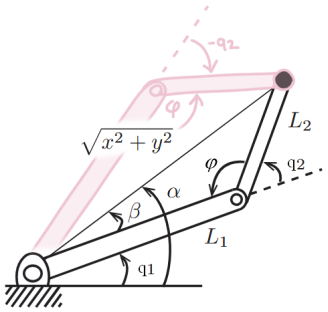
Using the slides from chapter 4: Forward Kinematics





After applying the law of cosine, we get:

$\alpha = \arctan(\frac{x_2}{x_1})$, $\beta = \arccos(\frac{l_1^2+x_1^2+x_2^2-l_2^2}{2l_1\sqrt{x_1^2+x_2^2}})$, $\varphi = \arccos(\frac{l_1^2+l_2^2-(x_1^2+x_2^2)}{2l_1l_2})$



But because we distinguish between elbow-up and elbow-down solutions, we have to also calculate their angles differently.

elbow-down
For q1 we simply subtract β from α.
For q2 we subtract φ from π.

elbow-up
For q1 we add instead of subtract.
For q2 we switch the order of minuend and subtrahend.

Implementation

Function

```
def inverse_kinematics_2dof(x, y, elbow_pos):
    # returning two angles, which generates the given TCP position
        in x and y

    # calculate alpha and beta
    alpha = math.atan2(y, x)
    beta = math.acos(( l1*l1 + x*x + y*y - l2*l2 ) / (2*l1*math.
        sqrt(x*x + y*y)))

    # calculate angle phi
    numerator = (pow(l1, 2) + pow(l2, 2) - (pow(x, 2) + pow(y, 2)))
    denominator = (2 * l1 * l2)
    phi = math.acos(numerator/denominator)

    # calculate angle q1 and q2 based on elbow position
    q1, q2 = 0, 0
    if elbow_pos == 'down':
        q2 = math.pi - phi
        q1 = alpha - beta
    elif elbow_pos == 'up':
        q2 = phi - math.pi
        q1 = alpha + beta

    return q1, q2
```

Calculating joints

```
tcp_pos = sim.getObjectPosition(sim.getObjectHandle("./TCP"), -1)
print("initial tcp pos: ", tcp_pos[0], tcp_pos[1])
try:
    tcp_pos_x = tcp_pos[0]+dx[0]
    tcp_pos_y = tcp_pos[1]+dx[1]
    beta = math.acos((l1 * l1 + tcp_pos_x * tcp_pos_x + tcp_pos_y *
         tcp_pos_y - l2 * l2) /
                     (2 * l1 * math.sqrt(tcp_pos_x * tcp_pos_x +
                         tcp_pos_y * tcp_pos_y)))
except ValueError:
    sim.stopSimulation()
    sys.exit("Error! TCP outside of workspace or singularity
        reached.")
else:
    if elbow_up:
        joints[0], joints[1] = inverse_kinematics_2dof(tcp_pos[0]+
            dx[0], tcp_pos[1]+dx[1], 'up')
    else:
        joints[0], joints[1] = inverse_kinematics_2dof(tcp_pos[0] +
             dx[0], tcp_pos[1] + dx[1], 'down')
```

Explanation

The function implements that which has been shown in the preparation.

The other code fragment shows how the joint angles are calculated.
Due to the fact that the TCP can be moved anywhere, there are two problem cases that can occur.
a) the TCP is outside of the robot's workspace, meaning it is impossible to reach.
b) the joints are aligned causing singularity, which leads to the loss of DOF.

For this reason we first check if math.acos(...) throws an error. This occurs if the input is outside of [-1,1].
This can be either due to a) or b). (The latter is testable this way because of numerical precision exceeding 1 due to rounding.)

If case a) or b) occur then the simulation ends and prints an error message. Otherwise, depending on the current value of the elbow_up flag, the solution for elbow_up or elbow_down are calculated.

Screenshots

```
initial tcp pos:  0.4264342659761567 0.24620193825300163
q1:  19.999999999944986
q2:  20.000000000107406
changed tcp pos:  0.42643426597609185 0.2462019382529512

elbow up:  1

initial tcp pos:  0.42643426597609185 0.2462019382529512
q1:  40.00000000010481
q2:  -20.00000000021483
changed tcp pos:  0.4264342659761049 0.24620193825288306
```

Figure 2: Program output. 1) elbow-down, 2) elbow-up

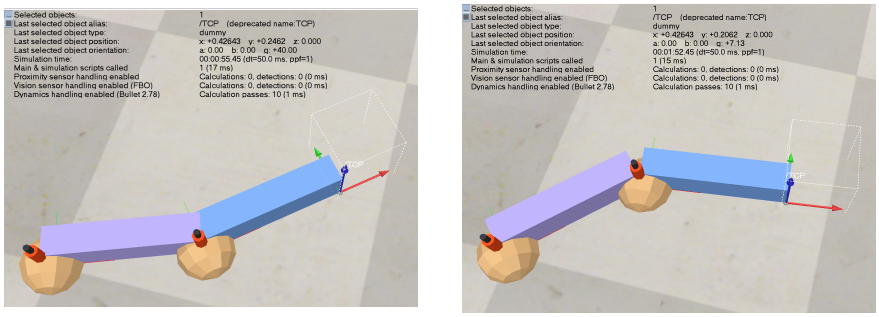


Figure 3: Both elbow positions. TCP stays the same.

Task 2

Build a 3-DOF robot consisting of 3 rotational joints, rotating around the z-axis of the world coordinate system (WCS). You can use the snake robot and downgrade it to 3 dofs or you build from scratch. The following figure shows the 3-DOF robot in its zero position (all joints are zero).

You can use one of your python scripts from exercise 2 as a starting point and calculate the inverse kinematics as a separate function.

This function should provide a numerical solution calculating the Jacobian matrix of the robot using partial derivatives. Use the **np.linalg.pinv(m)** function to compute the Moore-Penrose pseudo inverse of the Jacobian matrix.
At the start set all joints to 20 degrees to avoid singularities.
Control your robot's TCP in Cartesian space with your keyboard (see task 1).

Be aware, that this is a differential method based on linearization, so choose your values to steer the TCP carefully.

Alternative (voluntary): Evaluate your result with a small GUI (adopted from exercise 1) to control the TCP in Cartesian space.

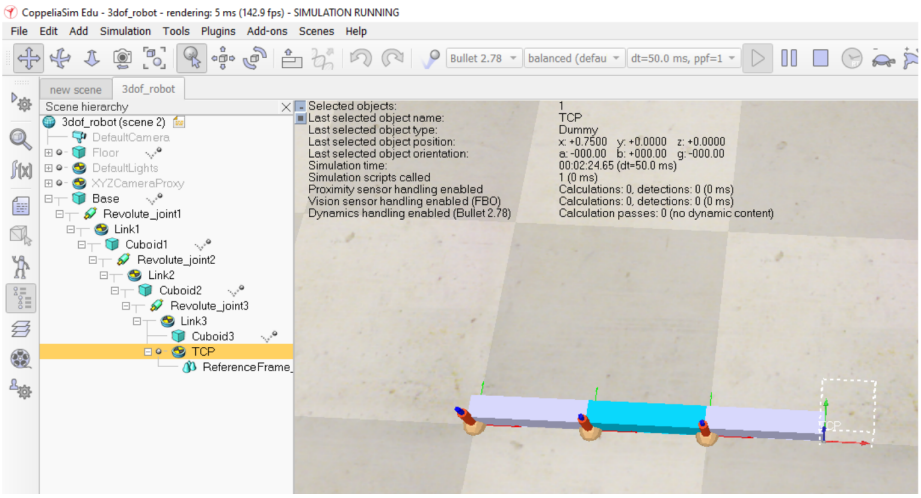
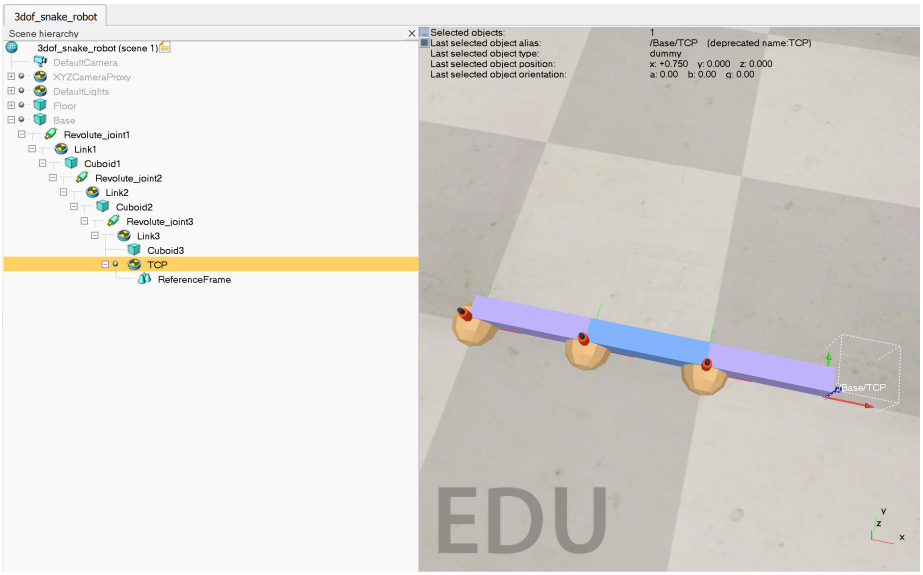


Figure 4: 3-DOF snake robot

3-DOF snake robot model



Made in CoppeliaSim 4.9.0 (rev. 6)

Preparation

Jacobian Calculation for $f(q_1, q_2, q_3)$

Given the functions:

$$f_1(q_1, q_2, q_3) = l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3)$$
$$f_2(q_1, q_2, q_3) = l_1 \sin(q_1) + l_2 \sin(q_1 + q_2) + l_3 \sin(q_1 + q_2 + q_3)$$
$$f_3(q_1, q_2, q_3) = q_1 + q_2 + q_3$$

We compute the Jacobian matrix $J = \frac{\partial \mathbf{f}}{\partial \mathbf{q}}$:

$$J = \begin{bmatrix} \frac{\partial f_1}{\partial q_1} & \frac{\partial f_1}{\partial q_2} & \frac{\partial f_1}{\partial q_3} \\ \frac{\partial f_2}{\partial q_1} & \frac{\partial f_2}{\partial q_2} & \frac{\partial f_2}{\partial q_3} \\ \frac{\partial f_3}{\partial q_1} & \frac{\partial f_3}{\partial q_2} & \frac{\partial f_3}{\partial q_3} \end{bmatrix}$$

Partial derivatives:

$$\frac{\partial f_1}{\partial q_1} = -l_1 \sin(q_1) - l_2 \sin(q_1 + q_2) - l_3 \sin(q_1 + q_2 + q_3),$$
$$\frac{\partial f_1}{\partial q_2} = -l_2 \sin(q_1 + q_2) - l_3 \sin(q_1 + q_2 + q_3),$$
$$\frac{\partial f_1}{\partial q_3} = -l_3 \sin(q_1 + q_2 + q_3),$$

$$\frac{\partial f_2}{\partial q_1} = l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3),$$
$$\frac{\partial f_2}{\partial q_2} = l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3),$$
$$\frac{\partial f_2}{\partial q_3} = l_3 \cos(q_1 + q_2 + q_3),$$

$$\frac{\partial f_3}{\partial q_1} = 1, \quad \frac{\partial f_3}{\partial q_2} = 1, \quad \frac{\partial f_3}{\partial q_3} = 1.$$

Thus, the Jacobian matrix is:

$$J = \begin{bmatrix} -l_1 \sin(q_1) - l_2 \sin(q_1 + q_2) - l_3 \sin(q_1 + q_2 + q_3) & -l_2 \sin(q_1 + q_2) - l_3 \sin(q_1 + q_2 + q_3) & -l_3 \sin(q_1 + q_2 + q_3) \\ l_1 \cos(q_1) + l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3) & l_2 \cos(q_1 + q_2) + l_3 \cos(q_1 + q_2 + q_3) & l_3 \cos(q_1 + q_2 + q_3) \\ 1 & 1 & 1 \end{bmatrix}$$

Implementation

Function

```
def inverse_jacobian_3dof(joint):
    # returning the inverse of the jacobian matrix
    q1 = joint[0]
    q2 = joint[1]
    q3 = joint[2]

    # First row of the matrix
    r00 = - l1*math.sin(q1) - l2*math.sin(q1 + q2) - l3*math.sin(q1
          + q2 + q3)
    r01 = - l2*math.sin(q1 + q2) - l3*math.sin(q1 + q2 + q3)
    r02 = - l3*math.sin(q1 + q2 + q3)

    # Second row of the matrix
    r10 = l1*math.cos(q1) + l2*math.cos(q1 + q2) + l3*math.cos(q1 +
          q2 + q3)
    r11 = l2*math.cos(q1 + q2) + l3*math.cos(q1 + q2 + q3)
    r12 = l3*math.cos(q1 + q2 + q3)

    # Third row of the matrix
    r20 = 1
    r21 = 1
    r22 = 1

    jacobian = np.array([[r00, r01, r02],
                         [r10, r11, r12],
                         [r20, r21, r22]])

    inv_jacobian = np.linalg.pinv(jacobian)

    return inv_jacobian
```

Calculating joints

```
inverse_jacobian = inverse_jacobian_3dof(joints)
# x and y coords of tcp
dx_tcp = np.array([dx[0], dx[1], 0.0])
dq = inverse_jacobian @ dx_tcp

# set new joint angles
joints[0] += dq[0]
joints[1] += dq[1]
joints[2] += dq[2]
```

Explanation

First, the joint angles are extracted from the parameter joint. Then the Jacobian from the preparation is simply copied over into the body and saved as a 2d array. At last we calculate the Moore-Penrose pseudo-inverse Jacobian using np.linalg.pinv() and return it.

When calculating the new joint angles we simply do the following:

$$d\theta \approx J^{-1}(\theta)dx$$

$$\theta_{new} = \theta_{old} + d\theta$$

We multiply the inverse Jacobian with the changes of the TCP pose.
This gives us the changes for the joints, which we add to the old angle resulting in the new, updated angle.

Screenshots

```
initial tcp pos:  0.5514342659761273 0.46270828919910345
jacobian:
 [[  8.95905645   7.51754097   1.        ]
 [-19.94896613 -11.51754097  -2.87938524]
 [ 10.98990968   4.           2.87938524]]
dx:  [0. 0. 0.]
dq:  [0. 0. 0.]
q1:  20.0
q2:  20.0
q3:  20.0
changed tcp pos:  0.5514342659761273 0.46270828919910345
```

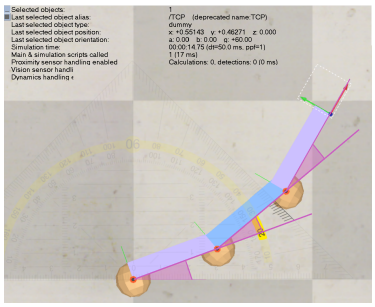


Figure 5: Initial (20,20,20) position

```
initial tcp pos:  0.5514342659761273 0.46270828919910345
jacobian:
 [[  8.95905645   7.51754097   1.        ]
 [-19.94896613 -11.51754097  -2.87938524]
 [ 10.98990968   4.           2.87938524]]
dx:  [ 0.         -1.14591559  0.        ]
dq:  [-8.61446739 13.19812975 -4.58366236]
q1:  11.385532606300796
q2:  33.19812975474582
q3:  15.416337638953387
changed tcp pos:  0.5481368213651794 0.44134630229387756
```

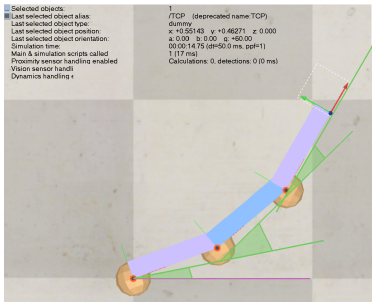


Figure 6: New angles (11,33,15) drawn using a protractor triangle

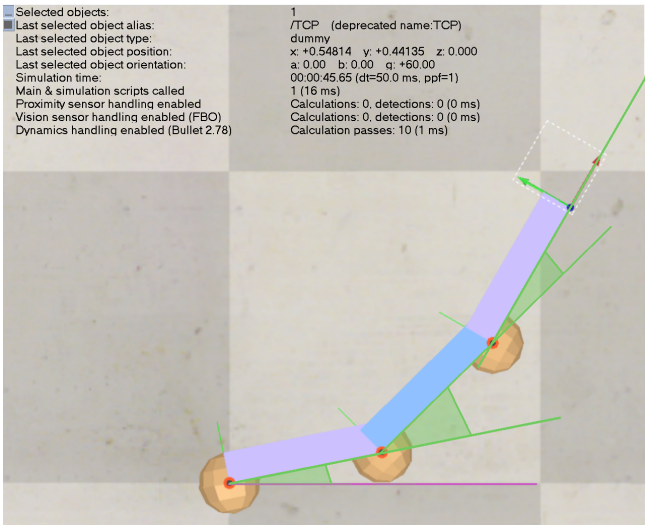


Figure 7: Comparing drawn angles with simulation

Task 3 (Voluntary)

Perform task 2 for your snake robot from practical exercise 2.
Alternatively, calculate the Jacobian by numerical differentiation for the snake robot.